

Appendix

Appendix A.

Derivation of focusing NLSE

For the sake of simplicity, whilst also maintaining the important aspects of solitons for analysis, it is useful to define a new system of units. First note from equation (??) that

$$g = \frac{2\hbar^2 a_s}{m, l_\perp^2}, \quad (1)$$

and since the focus of this paper is on bright solitons, take $g < 0$. The dimensionally reduced Gross-Pitaevskii equation (GPE) is then

$$i\hbar\psi_t = -\frac{\hbar^2}{2m}\psi_{xx} + g|\psi|^2\psi. \quad (2)$$

Here, ψ_t and ψ_{xx} denote the first temporal and second spatial derivatives, respectively. Now introduce the dimensionless variables.

$$x = x_0\hat{x}, \quad \psi = A_0\hat{\psi}, \quad t = t_0\hat{t}.$$

Substituting these into equation (2), and using

$$\psi_t = \frac{A_0}{t_0}\hat{\psi}_t, \quad \psi_{xx} = \frac{A_0}{x_0^2}\hat{\psi}_{xx}, \quad (3)$$

The following result is obtained

$$i\hbar\frac{A_0}{t_0}\hat{\psi}_t = -\frac{\hbar^2}{2m}\frac{A_0}{x_0^2}\hat{\psi}_{xx} + gA_0^3|\hat{\psi}|^2\hat{\psi}. \quad (4)$$

Dividing by the constant A_0 gives

$$i\hbar\frac{1}{t_0}\hat{\psi}_t = -\frac{\hbar^2}{2m}\frac{1}{x_0^2}\hat{\psi}_{xx} + gA_0^2|\hat{\psi}|^2\hat{\psi}. \quad (5)$$

Dividing through by $\hbar$ yields

$$i\hat{\psi}_t = -\frac{\hbar t_0}{2mx_0^2}\hat{\psi}_{xx} + \frac{gA_0^2 t_0}{\hbar}|\hat{\psi}|^2\hat{\psi}. \quad (6)$$

Choosing

$$t_0 = \frac{mx_0^2}{\hbar}, \quad \frac{gA_0^2 t_0}{\hbar} = -1,$$

reduces equation (6) to the focusing nonlinear Schrödinger equation (NLSE):

$$i\psi_t = -\frac{1}{2}\psi_{xx} - |\psi|^2\psi. \quad (7)$$

Using these simplifications on equation (??), the bright soliton solution becomes

$$\psi(x, t) = e^{iux} \text{sech}(x - x_0). \quad (8)$$

Appendix B.

Code for method comparison calculations

Listing 1: Efficiency comparison between explicit Euler, implicit Euler, Crank-Nicolson, Semi-implicit finite difference and the split step Fourier method

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import time
4
5 def soliton(x,t,v=0):
6     return np.exp(1j*(v*x/2-(v**2/4)*t)) * 1/np.cosh(x-v*t)
7
8 def laplacian(N, dx):
9     diag = -2*np.ones(N)
10    off = np.ones(N-1)
11    L = (np.diag(diag) + np.diag(off,1) + np.diag(off, -1)) / dx**2
12    return L
13
14 # =====
15 # ALL METHODS
16 # =====
17 def explicit_euler(psi, L, dt):
18     return psi + dt * 1j * (L @ psi + np.abs(psi)**2 * psi)
19
20 def split_step_fourier(psi, L, dt):
21     # Use global dx and N
22     global dx, N
23
24     # Wavenumbers for FFT
25     k = np.fft.fftfreq(N, d=dx) * 2*np.pi

```

```

26
27     # Nonlinear half-step
28     psi = psi * np.exp(1j * np.abs(psi)**2 * dt/2)
29
30     # Linear full-step in Fourier space
31     psi_hat = np.fft.fft(psi)
32     psi_hat = psi_hat * np.exp(-1j * (k**2) * dt)
33     psi = np.fft.ifft(psi_hat)
34
35     # Nonlinear half-step
36     psi = psi * np.exp(1j * np.abs(psi)**2 * dt/2)
37
38     return psi
39
40
41 def implicit(psi, L, dt):
42     N = len(psi)
43     A = np.eye(N) - dt * 1j * L
44     rhs = psi + dt * 1j * np.abs(psi)**2 * psi
45     return np.linalg.solve(A, rhs)
46
47 def crank_nicolson(psi, L, dt):
48     N = len(psi)
49     A = np.eye(N) + 0.5j*dt*L
50     B = np.eye(N) - 0.5j*dt*L
51     rhs = B @ psi + dt * 1j * np.abs(psi)**2 * psi
52     return np.linalg.solve(A, rhs)
53
54 def SIFD(psi, L, dt):
55     psi_mid = psi.copy()
56     for _ in range(3):
57         rhs = L @ psi_mid + np.abs(psi_mid)**2 * psi_mid
58         psi_mid = psi + 0.5*dt*1j*rhs
59     rhs_final = L @ psi_mid + np.abs(psi_mid)**2 * psi_mid
60     return psi + dt * 1j * rhs_final
61
62 def simulate(method, psi0, L, dt, steps):
63     psi = psi0.copy()
64     for _ in range(steps):

```

```

65     psi = method(psi, L, dt)
66     return psi
67
68 def error(psi_num, psi_exact):
69     return np.max(np.abs(psi_num - psi_exact))
70
71 #Define constant terms
72 Lx = 20
73 N = 400
74 dx = 2*Lx/(N-1)
75 x = np.linspace(-Lx, Lx, N)
76 dt = 0.001
77 T = 1
78 steps = int(T/dt)
79
80 Lmat = laplacian(N, dx)
81
82 #Stationary soliton
83 psi0 = soliton(x, 0)
84
85 methods = {
86     "Explicit Euler": explicit_euler,
87     "Implicit Euler": implicit,
88     "Crank-Nicolson": crank_nicolson,
89     "SIFD": SIFD,
90     "SSFM": split_step_fourier
91 }
92
93 print("Method comparison:\n")
94
95 #Loop over all methods to simulate accuracy and calculate the time taken
    /error
96 for name, method in methods.items():
97     start = time.time()
98     psi_num = simulate(method, psi0, Lmat, dt, steps)
99     runtime = time.time() - start
100
101     psi_exact = soliton(x, T)
102     err = error(psi_num, psi_exact)

```

```

103
104     print(f"{name:15s} | Error = {err:.3e} | Time = {runtime:.3f}s")

```

Appendix C.

Listing 2: Code used for density distributions, contour plots and error analysis of single, breathing, moving and colliding soliton solutions.

```

1
2  import numpy as np
3  import scipy.sparse as sp
4  import matplotlib.pyplot as plt
5  import scipy.sparse.linalg as spla
6  from scipy.signal import find_peaks
7  from mpl_toolkits.mplot3d import Axes3D
8
9  # Parameters
10 g = -1 #Interacion strength
11 L = 20 #Spatial domain length
12 Nx = 300 #Spatial points
13 T = 5 #Evolution time
14 Nt = 6000 #Time steps
15
16 # GPE constants
17 alpha = 1
18 beta = 0.5
19 gamma = -1
20
21 # Create the discretised spacetime grid:
22 x = np.linspace(-L/2,L/2,Nx)
23 dx = x[1]-x[0]
24
25 t = np.linspace(0,T,Nt)
26 dt = t[1]-t[0]
27
28 # From Taylor series create tridiagonal matrix
29 main_diag = -2 * np.ones(Nx)
30 upper_diag = 1*np.ones(Nx-1)

```

```

31 lower_diag = 1*np.ones(Nx-1)
32
33 A = sp.diags([lower_diag, main_diag, upper_diag],[-1,0,1])/dx**2
34
35 # Linear operator
36 Lop = -beta * A
37 I = sp.eye(Nx, format= "csc")
38
39 M_left = I+0.5j*dt*Lop
40 M_right = I-0.5j*dt*Lop
41
42
43 # Define all psi solutions
44 # =====
45 soliton_type = "single"
46 # =====
47 plt.rcParams['xtick.labelsize'] = 12
48 plt.rcParams['ytick.labelsize'] = 12
49 psi = np.zeros((Nt,Nx), dtype =complex)
50
51 # Initial
52 if soliton_type == "single":
53     psi[0,:] = 1/np.cosh(x)
54
55 # Breathing
56 elif soliton_type == "breathing":
57     psi[0,:] = 2/np.cosh(x)
58
59 # Moving
60 elif soliton_type=="moving":
61     psi[0,:] = np.exp(4j*2*np.pi*x/L)*1/np.cosh(x)
62
63 # Colliding
64 elif soliton_type=="colliding":
65     #Define a moving soliton
66     def moving_soliton(x, x0, v, phase=0):
67         return np.exp(1j*(v*x+phase))*(1/np.cosh(x-x0))
68
69     # Set initial conditions for both solitons

```

```

70     x1, v1 = -5, 2 # Soliton moving right
71     x2, v2 = 5, -2 # Soliton moving left
72     phi = 0 # phase
73     phases = [np.pi/2, np.pi, 3*np.pi/2, np.pi*2]
74     psi[0,:] = (moving_soliton(x, x1, v1, phase = 0)+moving_soliton(x,
75                               x2, v2, phase = phi))
76
77 # Store density values to eventually plot as contour
78 density = np.zeros((Nt,Nx))
79 density[0,:] = np.abs(psi[0,:])**2
80 if soliton_type == "breathing":
81     peaks = np.max(density)
82
83 # Time evolution
84 peak_over_time = np.zeros(Nt)
85 peak_over_time[0] = np.max(density[0])
86
87 psi_n = psi[0,:].copy()
88
89 # Particle number tracking
90 # Create the norm to track
91 N_t = np.zeros(Nt)
92 N_t[0] = np.sum(np.abs(psi[0, :])**2) * dx
93
94 # Semi-implicit method, evolve wavefunction with time
95 # Pre-factorise matrix once
96 M_left_factor = spla.factorized(M_left)
97
98 for n in range(1, Nt):
99     # Use previous timestep as first guess
100     psi_new = psi_n.copy()
101
102     # 3 Midpoint iterations as it converges
103     for k in range(3):
104         psi_mid = 0.5 * (psi_n + psi_new)
105         nonlinear_mid = -1j * dt * gamma * np.abs(psi_mid)**2 * psi_mid
106         rhs = M_right @ psi_n + nonlinear_mid
107         psi_new = M_left_factor(rhs)

```

```

108
109     psi_n = psi_new.copy()
110     psi[n, :] = psi_n
111     density[n, :] = np.abs(psi_n)**2
112     peak_over_time[n] = np.max(density[n])
113     # Update the norm track it through the evolution
114     N_t[n] = np.sum(np.abs(psi_n)**2) * dx
115
116 # Plot
117 if soliton_type == "breathing":
118     plt.plot(t, peak_over_time)
119     plt.xlabel("Time", fontsize=16)
120     plt.ylabel("Peak density", fontsize=16)
121     plt.title("Breathing soliton peak amplitude over time", fontsize=16)
122     plt.grid()
123     plt.show()
124
125     # Find local maxima of the peak signal
126     indices, _ = find_peaks(peak_over_time)
127     # Convert to times
128     t_peaks = t[indices] # Consecutive peak spacings
129     T_measured = np.diff(t_peaks)
130     print("Peak-to-peak periods:", T_measured)
131     print("Target period /2:", np.pi/2)
132     print("Relative error:", abs(T_measured - np.pi/2)/(np.pi/2))
133
134 plt.plot(x, np.abs(psi_n)**2)
135 if soliton_type=="single":
136     plt.title("Density distribution of single soliton")
137 elif soliton_type == "breathing":
138     plt.title("Density distribution of breathing soliton")
139     final_peak = np.max(density[-1])
140     plt.plot(x, density[-1])
141     plt.axhline(final_peak, color='r', linestyle='--', label = "Peak")
142 elif soliton_type == "moving":
143     plt.title("Density distribution of moving soliton")
144 elif soliton_type == "colliding":
145     plt.title("Density distribution of two moving, interacting solitons"
146             )

```



```

146
147 plt.legend()
148 plt.xlabel("x")
149 plt.ylabel(r"Density,  $\psi^2$ ")
150 plt.show()
151
152 # Contour plot of single bright soliton
153 X, T = np.meshgrid(x,t)
154 Z = density
155
156
157
158 plt.contourf(X,T,density,levels = 50, cmap = 'gnuplot2')
159 colcol = plt.colorbar(label = r"Density of soliton  $\psi^2$ ")
160 colcol.set_label(label = r"Density of soliton  $\psi^2$ ",
    fontsize=14, labelpad = 15)
161 plt.xlabel("x", fontsize = 14)
162 plt.ylabel("Time evolution, t", fontsize = 14)
163
164
165
166 psi_exact = np.exp(4j*2*np.pi*x/L)*1/np.cosh(x)
167 psi_numerical = psi[-1,:]
168 error = np.max(np.abs(psi_numerical - psi_exact))
169
170 # =====
171 # Colliding solitons comparison (Extension)
172 # =====
173 # For the colliding solitons, the same steps are taken from above, I
    have looped it to see how
174 # phase changes interaction
175 plt.rcParams['xtick.labelsize'] = 30
176 plt.rcParams['ytick.labelsize'] = 30
177
178 labels = [r' $\phi = \pi/2$ ', r' $\phi = \pi$ ', r' $\phi = 3\pi/2$ ', r' $\phi = 2\pi$ ']
179
180 if soliton_type == "colliding":
181     all_densities = []

```

```

182 Theo_dens = []
183
184 phases = [0, np.pi/2, np.pi, 3*np.pi/2, 2*np.pi]
185
186 for phi in phases:
187     psi_col = np.zeros((Nt,Nx), dtype = complex)
188     density_col = np.zeros((Nt,Nx))
189
190     #Set initial conditions for both solitons
191     x1, v1 = -5, 2 # Soliton moving right
192     x2, v2 = 5, -2 # Soliton moving left
193
194     psi_col[0,:] = (moving_soliton(x, x1, v1, phase = 0)+
195                     moving_soliton(x, x2, v2, phase = phi))
196     density_col[0,:] = np.abs(psi_col[0,:])**2
197
198     psi_col_n = psi_col[0,:].copy()
199
200     M_left_factor = spla.factorized(M_left)
201
202     for n in range(1, Nt):
203         psi_new_col = psi_col_n.copy()
204
205         for k in range(3):
206             psi_mid_col = 0.5 * (psi_col_n + psi_new_col)
207             nonlinear_mid_col = -1j * dt * gamma * np.abs(
208                 psi_mid_col)**2 * psi_mid_col
209             rhs_col = M_right @ psi_col_n + nonlinear_mid_col
210             psi_new_col = M_left_factor(rhs_col)
211
212             psi_col_n = psi_new_col.copy()
213             psi_col[n,:] = psi_col_n
214             density_col[n,:] = np.abs(psi_col_n)**2
215
216     # Theoretical density deviation calculation
217     Theoretical_density = 2*(1+np.cos(phi))
218     Theo_dens.append(Theoretical_density)
219     all_densities.append(density_col)

```

```

219
220 # Convergence test
221 psi_test = psi[0,:].copy()
222 psi_new_test = psi_test.copy()
223 print("-----")
224 print("Convergence test:")
225 for k in range(3):
226     psi_mid_test = 0.5 * (psi_test + psi_new_test)
227     psi_prev_test = psi_new_test.copy()
228     nonlinear_test = -1j * dt * gamma * np.abs(psi_mid_test)**2 *
        psi_mid_test
229     rhs_test = M_right @ psi_test + nonlinear_test
230     psi_new_test = M_left_factor(rhs_test)
231     conv = np.max(np.abs(psi_new_test - psi_prev_test))
232     print(f" k={k+1}: = {conv:.2e}")
233 print("-----")
234 x0_idx = np.argmin(np.abs(x))
235 numerical = [np.max(d[:, x0_idx]) for d in all_densities]
236 print("Phase | Theoretical | Numerical")
237 for phi, th, num in zip(phases, Theo_dens, numerical):
238     print(f"{phi:.4f} | {th:.4f} | {num:.4f}")
239 print("-----")
240 uncertainty = dx**2 * np.ones(len(phases))
241 phi_smooth = np.linspace(0, 2*np.pi, 500)
242
243 fig, axes = plt.subplots(2, 2, figsize=(25, 20), sharey=True)
244 X, Tgrid = np.meshgrid(x, t)
245 Z = np.abs(psi_col)**2
246 axes = axes.ravel()
247
248 contours = []
249 for ax, density_col, label in zip(axes, all_densities[1:], labels):
250     c = ax.contourf(X, Tgrid, density_col, levels=500, cmap='gnuplot2')
251     contours.append(c)
252     ax.set_title(f"Phase {label}", fontsize=30)
253     ax.set_xlabel("x", fontsize=30)
254
255 axes[0].set_ylabel("Time, t", fontsize=30)
256 axes[2].set_ylabel("Time, t", fontsize=30)

```

```

257
258
259 cbar_coll = fig.colorbar(c, ax=axes, location='right', shrink=0.8,
    pad=0.05)
260 cbar_coll.set_label(label=r"Soliton Density  $|\psi|^2$ ",
    fontsize=30, labelpad = 15)
261 cbar_coll.solids.set_rasterized(True)
262 cbar_coll.solids.set_edgecolor("face")
263 plt.show()
264
265 fig, ax = plt.subplots(figsize=(8, 5))
266
267 # Smooth theoretical curve
268 ax.plot(phi_smooth, 2*(1+np.cos(phi_smooth)), 'b-', linewidth=1.5,
    label=r'Theoretical:  $2A^2(1+\cos(\Delta\phi))$ ', zorder=1)
269
270 # Numerical points with error bars (the crosses)
271 ax.errorbar(phases, numerical, yerr=uncertainty, fmt='x', color='k',
    markersize=8, markeredgewidth=1, capsize=4, capthick=1, linewidth
    =1, label='Numerical simulation', zorder=2)
272
273 ax.set_xlabel(r'Global phase  $\Delta\phi$ ', fontsize=16)
274 ax.set_ylabel(r'Peak collision density  $|\psi|^2$ ', fontsize=16)
275 ax.set_title(r'Peak soliton collision density against imposed phase',
    fontsize=16)
276
277 # Exact phase tick labels
278 ax.set_xticks([0, np.pi/2, np.pi, 3*np.pi/2, 2*np.pi])
279 ax.tick_params(axis='both', labelsize=16)
280 ax.set_xticklabels([r'$0$', r'$\frac{\pi}{2}$', r'$\pi$', r'$\frac{3\pi}{2}$', r'$2\pi$'],
    fontsize=12)
281
282 ax.set_xlim(-0.1, 2*np.pi + 0.1)
283 ax.set_ylim(-0.3, 5)
284
285 ax.grid(True, linestyle='--', alpha=0.4, linewidth=0.8)
286 ax.legend(fontsize=11, framealpha=0.9)
287
288 plt.tight_layout()

```

```

289 plt.show()
290
291 # =====
292 # Error estimation
293 # =====
294 # L2 error and density errors using RMS
295 psi_exact = 1/np.cosh(x)
296 L2_error = np.sqrt(np.sum(np.abs(psi[-1,:] - psi_exact)**2)/Nx)
297 Dens_err = np.sqrt(np.sum(np.abs(psi[-1,:]**2 - psi_exact**2))/Nx)
298 print(f"L2 error = {L2_error:.8f}")
299 print(f"Dens error = {Dens_err:.8f}")
300 print("-----")
301 # Variation from the norm, conservation of particle test
302 N0 = N_t[0]
303 rel_dev = np.abs(N_t - N0) / N0
304 max_rel_dev = np.max(rel_dev)
305 print(f"Max relative norm deviation: {max_rel_dev:.2e}")
306 print("-----")
307
308 # =====
309 # 2nd order convergence test
310 # =====
311 def run_convergence_test(Nx, Nt):
312     # Rerun the code above in the definition
313     L = 20
314     T = 1.0
315
316     beta = 0.5
317     gamma = -1
318
319     x = np.linspace(-L/2, L/2, Nx)
320     dx = x[1] - x[0]
321     t = np.linspace(0, T, Nt)
322     dt = t[1] - t[0]
323
324     main_diag = -2 * np.ones(Nx)
325     upper_diag = np.ones(Nx-1)
326     lower_diag = np.ones(Nx-1)

```

```

327     A = sp.diags([lower_diag, main_diag, upper_diag], [-1, 0, 1]) / dx
328         **2
329
330     Lop = -beta * A
331     I = sp.eye(Nx, format="csc")
332     M_left = I + 0.5j * dt * Lop
333     M_right = I - 0.5j * dt * Lop
334     M_left_factor = spla.factorized(M_left)
335
336     # initial condition for stationary soliton
337     psi_n = 1/np.cosh(x)
338     psi = np.zeros((Nt, Nx), dtype=complex)
339     psi[0, :] = psi_n
340
341     for n in range(1, Nt):
342         psi_new = psi_n.copy()
343         for k in range(3):
344             psi_mid = 0.5 * (psi_n + psi_new)
345             nonlinear_mid = -1j * dt * gamma * np.abs(psi_mid)**2 *
346                 psi_mid
347             rhs = M_right @ psi_n + nonlinear_mid
348             psi_new = M_left_factor(rhs)
349             psi_n = psi_new.copy()
350             psi[n, :] = psi_n
351
352     psi_exact = 1/np.cosh(x)
353     dens_err = np.sqrt(np.sum((np.abs(psi[-1, :])**2 - np.abs(psi_exact)
354         **2)**2) / Nx)
355     return dens_err
356
357
358 Nx_list = [150, 300, 600]
359 errors = []
360
361 for Nx in Nx_list:
362     Nt = 6000 * Nx_list[1] // Nx
363     err = run_convergence_test(Nx, Nt)
364     errors.append(err)

```

```
363 for i in range(1, len(errors)):
364     ratio = errors[i-1] / errors[i]
365     print(f"E_{i}/E_{i+1} = {ratio:.2f}")
366 print("-----")
```